

PROJETO PEDIDO VELOZ

Cloud DevOps: Orchestrating Containers
and Microservices

Joice Barbosa Santos

RA: 102859

ADSNA – 5º Semestre

Repositório GitHub:

<https://github.com/JoiceBsantos/pedido-veloz-devops>

Taboão da Serra - SP

2026

SUMÁRIO

1 INTRODUÇÃO	3
2 FUNDAMENTAÇÃO TEÓRICA	3
2.1 Arquitetura de Microsserviços	3
2.2 Docker e Containerização	3
2.3 Kubernetes	4
2.4 CI/CD e Observabilidade	4
3 DESENVOLVIMENTO	4
3.1 Estrutura do Projeto	5
3.2 Docker Compose e Containers	5
3.3 Microsserviços e APIs	6
3.4 Kubernetes	7
3.5 Terraform e Observabilidade	7
4 RESULTADOS	8
5 CONCLUSÃO	9
REFERÊNCIAS	9

1 INTRODUÇÃO

O projeto Pedido Veloz foi desenvolvido para a disciplina de Cloud DevOps com o objetivo de implementar uma arquitetura cloud-native baseada em microsserviços. O cenário proposto simulava a modernização da plataforma de uma empresa de e-commerce chamada Loja Veloz, que necessitava melhorar escalabilidade, disponibilidade, automação e confiabilidade operacional.

A solução foi construída utilizando Docker, Kubernetes, GitHub Actions, Terraform e ferramentas de observabilidade, aplicando conceitos modernos de DevOps e infraestrutura distribuída.

O projeto contemplou a implementação de microsserviços independentes, containerização com Docker, orquestração Kubernetes, automação CI/CD e estrutura de observabilidade baseada em Prometheus e Grafana.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Arquitetura de Microsserviços

A arquitetura de microsserviços permite dividir aplicações em serviços independentes, facilitando manutenção, escalabilidade e implantação. Cada serviço executa funções específicas e pode ser atualizado individualmente sem impactar os demais componentes do sistema.

No projeto Pedido Veloz foram implementados quatro microsserviços principais: API Gateway, pedidos-service, pagamentos-service e estoque-service, permitindo modularidade e desacoplamento da aplicação.

2.2 Docker e Containerização

O Docker foi utilizado para containerizar os microsserviços da aplicação, garantindo portabilidade, isolamento e padronização do ambiente de execução. A utilização de containers facilita a distribuição e implantação da aplicação em diferentes ambientes computacionais.

Além disso, foi utilizado Docker Compose para gerenciamento simultâneo dos containers da aplicação, permitindo inicialização centralizada dos serviços.

2.3 Kubernetes

O Kubernetes foi utilizado como plataforma de orquestração de containers, permitindo gerenciamento automatizado de deployments, escalabilidade horizontal, serviços e balanceamento de carga.

No projeto foram implementados Deployments, Services, ReplicaSets, ConfigMaps, Secrets e Horizontal Pod Autoscaler (HPA), simulando uma arquitetura cloud-native moderna baseada em microsserviços.

A utilização do Kubernetes proporcionou maior controle sobre os containers da aplicação, permitindo gerenciamento centralizado e preparação para ambientes distribuídos.

2.4 CI/CD e Observabilidade

O projeto utilizou GitHub Actions para estruturação da pipeline CI/CD, permitindo automação de processos de integração contínua e organização do fluxo de desenvolvimento.

Também foi preparada uma estrutura de observabilidade utilizando Prometheus e Grafana, possibilitando monitoramento da aplicação e coleta de métricas em ambientes distribuídos.

Além disso, foram utilizados conceitos de Infrastructure as Code (IaC) através do Terraform, permitindo gerenciamento padronizado da infraestrutura da aplicação.

3 DESENVOLVIMENTO PRÁTICO

3.1 Estrutura do Projeto

O projeto foi organizado em módulos independentes, contendo microsserviços, arquivos Docker, manifests Kubernetes, estrutura de observabilidade, pipeline CI/CD e infraestrutura como código utilizando Terraform.

```
pedido-veloz-devops/  
├── .github/workflows/  
├── api-gateway/  
├── pedidos-service/  
├── pagamentos-service/  
├── estoque-service/  
├── k8s/  
├── observability/  
├── terraform/  
├── docs/  
│   └── imagens/  
├── docker-compose.yml  
├── README.md  
└── LICENSE
```

Figura 1 — Estrutura organizacional do projeto Pedido Veloz DevOps

3.2 Docker Compose e Containers

A aplicação foi executada utilizando Docker Compose, permitindo inicialização centralizada dos microsserviços e do banco de dados PostgreSQL através de containers independentes.

Foram criados containers para API Gateway, pedidos-service, pagamentos-service, estoque-service e PostgreSQL, garantindo isolamento dos serviços e padronização do ambiente de execução.

```
pedidos-service-1 | > pedidos-service@1.0.0 start
api-gateway-1 | > api-gateway@1.0.0 start

pedidos-service-1 | > node index.js
api-gateway-1 | > node index.js
pedidos-service-1 |
api-gateway-1 |
pagamentos-service-1 | > pagamentos-service@1.0.0 start
pagamentos-service-1 | > node index.js
estoque-service-1 | > estoque-service@1.0.0 start
estoque-service-1 | > node index.js
estoque-service-1 |
api-gateway-1 | API Gateway rodando na porta 3000
pedidos-service-1 | Pedidos Service rodando na porta 3001
estoque-service-1 | Estoque Service rodando na porta 3003
pagamentos-service-1 | Pagamentos Service rodando na porta 3002

View in Docker Desktop View Config Enable Watch Detach
```

Figura 2 — Execução dos microsserviços utilizando Docker Compose

3.3 Microsserviços e APIs

Os microsserviços foram executados individualmente através de containers Docker, disponibilizando endpoints HTTP locais para validação da arquitetura distribuída.

Foram implementados serviços independentes para gerenciamento de pedidos, pagamentos e estoque, além de um API Gateway responsável pelo roteamento centralizado das requisições da aplicação.

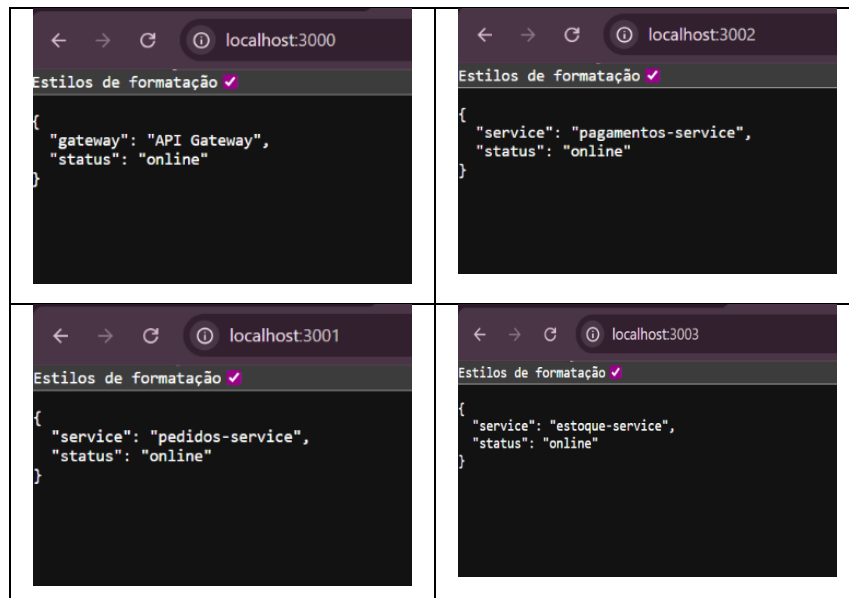


Figura 3 — Microsserviços e API Gateway executando localmente

3.4 Kubernetes

Após a execução dos containers via Docker Compose, a aplicação também foi preparada para orquestração utilizando Kubernetes.

Foram criados manifests YAML contendo Deployments, Services, ConfigMaps, Secrets e Horizontal Pod Autoscaler (HPA), permitindo gerenciamento centralizado da aplicação e simulação de escalabilidade horizontal em ambiente cloud-native.

A execução local do cluster Kubernetes foi realizada utilizando Docker Desktop com Kubernetes habilitado.

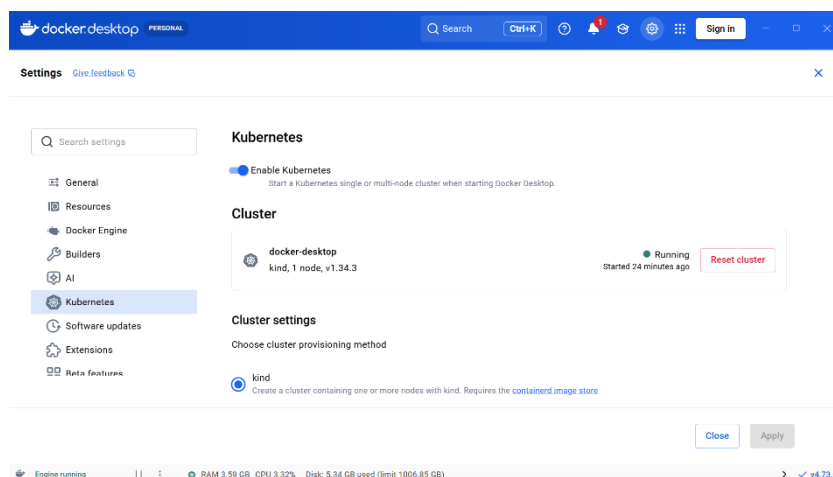


Figura 4 — Cluster Kubernetes executando no Docker Desktop

3.5 Terraform e Observabilidade

O projeto também contemplou conceitos de Infrastructure as Code (IaC) utilizando Terraform para organização da infraestrutura da aplicação.

Além disso, foi preparada uma estrutura de observabilidade utilizando Prometheus e Grafana, permitindo monitoramento da aplicação e coleta de métricas em ambientes distribuídos.

As configurações de observabilidade foram organizadas em diretórios independentes, seguindo práticas modernas de arquitetura cloud-native.

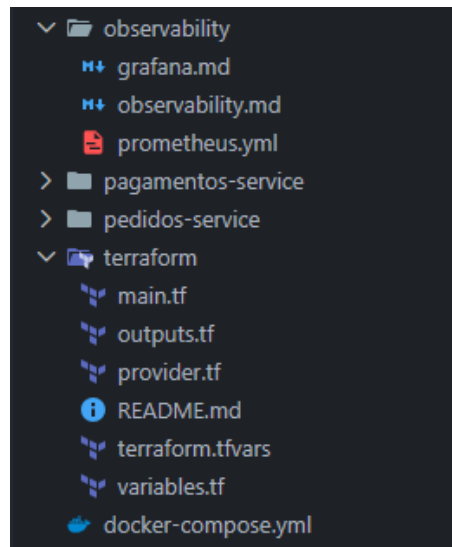


Figura 5 — Estrutura de observabilidade e Infrastructure as Code

4 RESULTADOS

Os resultados obtidos demonstraram o funcionamento correto da arquitetura cloud-native implementada no projeto Pedido Veloz.

Os microsserviços foram executados em containers Docker, permitindo isolamento dos serviços e padronização do ambiente de execução. A utilização do Docker Compose possibilitou gerenciamento centralizado da aplicação e inicialização simultânea dos containers.

Também foi implementada estrutura Kubernetes contendo Deployments, Services, ConfigMaps, Secrets e Horizontal Pod Autoscaler (HPA), simulando um ambiente moderno de orquestração de containers.

Além disso, o projeto contemplou pipeline CI/CD utilizando GitHub Actions, estrutura de observabilidade baseada em Prometheus e Grafana e organização de infraestrutura como código utilizando Terraform.

5 CONCLUSÃO

O desenvolvimento do projeto Pedido Veloz possibilitou aplicar conceitos modernos de Cloud DevOps, incluindo microsserviços, containerização, orquestração Kubernetes, automação CI/CD e observabilidade.

A implementação permitiu compreender práticas utilizadas em arquiteturas cloud-native modernas, proporcionando experiência prática com Docker, Kubernetes, GitHub Actions, Terraform, Prometheus e Grafana.

O projeto demonstrou a importância da automação, escalabilidade e modularização de aplicações distribuídas, alinhando conhecimentos teóricos e práticos da disciplina de Cloud DevOps.

REFERÊNCIAS

DOCKER. Docker Documentation. Disponível em: <https://docs.docker.com/>. Acesso em: 12 maio 2026.

KUBERNETES. Kubernetes Documentation. Disponível em: <https://kubernetes.io/docs/>. Acesso em: 12 maio 2026.

HASHICORP. Terraform Documentation. Disponível em: <https://developer.hashicorp.com/terraform/docs>. Acesso em: 12 maio 2026.

GITHUB. GitHub Actions Documentation. Disponível em: <https://docs.github.com/actions>. Acesso em: 12 maio 2026.

PROMETHEUS. Prometheus Documentation. Disponível em: <https://prometheus.io/docs/>. Acesso em: 12 maio 2026.

GRAFANA. Grafana Documentation. Disponível em: <https://grafana.com/docs/>. Acesso em: 12 maio 2026.